National University of Computer & Emerging Sciences

Lecture 23

Bakery Algorithm

Multiple Process Solutions

- Peterson's solution solves the critical-section problem for two processes, in software
- For multiple processes we have "Bakery Algorithm"
- Used in bakeries etc ?? No ☺

Bakery Algorithm

- The basic idea is that of a bakery
- On entering the bakery, Customers take tokens
- Whoever has the lowest token gets service next.
- “Service" means entry to the critical section.

Bakery Algorithm

- `int token[n];`
- `token[0]` = token given to Process 0
- `token[1]` = token given to Process 1
- …
- …
- `token[n-1]` = token given to Process n – 1

Receive a token

- Initially token[0] .. token[n-1] are set to zero
- Process i chooses token[i] as
  - `max(token[0],token[1],...,token[n-1]) + 1;`
- Let n = 5;
- Let the order of execution be P0,P3,P4,P1,P2,P3,P4…
- P0 gets token[0] = max(0,0,0,0,0) + 1 = 0+1=1
- P3 gets token[3] = max(1,0,0,0,0) + 1 = 1+1=2
- P4 gets token[4] = max(1,0,0,2,0) + 1 = 2+1=3
- P1 gets token[1] = max(1,0,0,2,3) + 1 = 3+1=4
- P2 gets token[2] = max(1,4,0,2,3) + 1 = 4+1=5
- P3 gets token[3] = max(1,4,5,2,3) + 1 = 5+1=6
- P4 gets token[4] = max(1,4,5,6,3) + 1 = 6+1=7

Algorithm

```
while(TRUE)
{
 //1. Receive a token
 token[OwnID]=    max(token[0],token[1],..,token[n-1])+1;
 //2. Wait for turn
 for (OthersID = 0;OthersID<n;OthersID++)
   while(token[OthersID]!=0 &&(token[OthersID],OthersID)<
   (token[OwnID],OwnID));
 //3. Enter Critical section
 critical_section();
 //4. Leave Critical Section
 token[OwnID] = 0;
}
```

Algorithm

```
while(TRUE)
{
 //1. Receive a token
 token[OwnID]=     max(tol                    token[n-1])+1;
 //2. Wait for turn
 for (OthersID = 0;OthersID<n;OthersID++)
   while(token[OthersID]!=0 &&(token[OthersID],OthersID)<
    (token[OwnID],OwnID));
 //3. Enter Critical section
 critical_section();
 //4. Leave Critical Section
 token[OwnID] = 0;
}
```

Why multiple waits?

Because, have to wait for multiple processes

Wait for turn

- Pi waits until it has the lowest token of all the processes waiting to enter the critical section.
- Bakery Algorithm does not guarantee that two processes do not receive the same token
- In case of a tie, the process with the lowest ID is served first.

```
for (OthersID = 0;OthersID < n ; OthersID++)
while(token[OthersID]!=0 &&(token[OthersID],OthersID)<
(token[OwnID],OwnID));
```

- (a,b) < (c,d) = TRUE if a < c or if both a = c and b < d
- token[OwnID] = 0 => Process is not trying to enter the critical section

Problem with 1st try

- *Suppose no process is in the critical section and token is set to 0*
- Process 1 wants to enter into critical section and getting the token.
- Process 1 computes the token (Max+1) which is token # 1 for itself and process 1 times out before it assigns the token to itself.
- Process 2 came and receive the token # 1
- Its token is lowest token at the moment so it gets into the CS & timeout
- Now Process 1 came again and assigned the already computed token to itself which is token # 1.
- Now process 1 has the same token number as process 2
- So 2nd condition become false for process 1 in the spinning loop since process 1 has lowest process ID (j<i ,where j and i are representing process ids)
- `(number[j],j)< (number[i],i))`
- Process 1 will also enter into the CS
- Hence Mutual Exclusion is voilated

Solution

1 Declare choosing status

- A process must declare its choosing status as true when it is choosing a token.
- It must declare choosing status as false once it is assigned with token.

2 **Wait and let other process to choose a token**

- Wait and spins when some other process has declared its choosing status as true before entering into the critical section loop.

Bakery Algorithm

- The reason for **`choosing`** is to prevent the second **while** loop being *entered* when process $P_{OthersID}$ is setting its **`token[OthersID]`**.
- **`choosing[OthersID]`** is true if $P_{OthersID}$ is choosing a token.
- If a process $P_{OthersID}$ is choosing a token when Pi tries to look at it, Pi waits until $P_{OthersID}$ has done so before looking

Bakery Algorithm

```
repeat
    choosing[i] := true;
    number[i] := max(number[0], number[1], ..., number[n - 1])+1;
    choosing[i] := false;
    for j := 0 to n - 1
        do begin
            while choosing[j] do no-op;
            while number[j] != 0
                and (number[j], j) < (number[i], i) do no-op;
        end;

        critical section

    number[i] := 0;

        remainder section

until false;
```

- Firstly the process sets its “choosing” variable to be TRUE indicating its intent to enter critical section.
- Then it gets assigned the highest ticket number corresponding to other processes.
- Then the “choosing” variable is set to FALSE indicating that it now has a new ticket number. It is actually a small critical section in itself
- First it waits while other process is choosing token
- Then it checks two conditions of all processes waiting for critical section
  - if any other process has a non zero token && its own token is less than from all tokens
  - If two process got same tokens- to break the tie – compare process id & a process with lower id will get the critical section.
- After that we proceed to check the ticket values of processes where process with least ticket number/process id gets inside the critical section.
- The exit section just resets the ticket value to zero.

Bakery Algorithm

- Works for any number of processes and is suitable for a distributed environment.
- Whenever a process wishes to enter its critical section, it chooses a number (as in a bakery). The algorithm guarantees that the number chosen is >= the number held by any other process waiting for or in its critical section. (It may be = if two processes choose at the same time, but this is resolved by using the process id to break the tie.)

bakery Algorithm proof

- A process may only enter its critical section if its number is the smallest among all the waiting processes - hence mutual exclusion is guaranteed.
- Furthermore, once a process arrives at this point, its entry is guaranteed, so deadlock is not possible.(progress)
- Finally, a process leaving the critical section and turning around to re-enter must get a number greater than than of any waiting process that has already chosen, so bounded wait is ensured.